

MODULE 18

Model Deployment and MLOps

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Model Deployment and MLOps

What is MLOps?

MLOps (Machine Learning Operations) bridges data science and software engineering.

ML Lifecycle:

1. Problem definition
2. Data collection and preparation
3. Model training and evaluation
4. Deployment
5. Monitoring
6. Retraining

Key tools: MLflow, DVC, Kubeflow, Airflow, Docker

Save and Load Models

```
import joblib, pickle
from sklearn.ensemble import RandomForestClassifier

# Train model
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Save with joblib (recommended for sklearn)
joblib.dump(model, 'model.pkl')
model_loaded = joblib.load('model.pkl')

# Save with pickle
with open('model.pkl', 'wb') as f:
    pickle.dump(model, f)

# TensorFlow/Keras
model.save('model.h5')
model = tf.keras.models.load_model('model.h5')
```

Flask API for ML Model

```
from flask import Flask, request, jsonify
import joblib, numpy as np

app = Flask(__name__)
model = joblib.load('model.pkl')
scaler = joblib.load('scaler.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.json['features']
```

```

X = np.array(data).reshape(1, -1)
X_scaled = scaler.transform(X)
prediction = model.predict(X_scaled)[0]
probability = model.predict_proba(X_scaled)[0].tolist()
return jsonify({'prediction': int(prediction), 'probability': probability})

if __name__ == '__main__':
    app.run(debug=True, port=5000)

```

MLflow Experiment Tracking

```

import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

mlflow.set_experiment('my_experiment')

with mlflow.start_run():
    model = RandomForestClassifier(n_estimators=100, max_depth=5)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    mlflow.log_param('n_estimators', 100)
    mlflow.log_param('max_depth', 5)
    mlflow.log_metric('accuracy', acc)
    mlflow.log_metric('f1_score', f1)
    mlflow.sklearn.log_model(model, 'model')

```

Docker for ML

Containerise ML models for reproducible deployment.

```

# Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]

# Build and run
# docker build -t ml-app .
# docker run -p 5000:5000 ml-app

# docker-compose.yml
# version: '3'
# services:
# ml-api:
# build: .
# ports:
# - '5000:5000'

```

